

```
import { useState, useEffect } from "react";
```

```
/* _____  
Berkeley Country Club – Redwood Tree Member Poll  
Design: club scorecard vernacular. Deep pine green, scorecard  
cream, redwood-bark accent, brass gold. Serif display set like  
a card header; results render as fairway-stripe tally bars.  
Responses persist in shared storage (visible to all members).  
_____ */
```

```
const C = {  
  pine: "#1E3D2B",  
  pineDeep: "#142B1E",  
  cream: "#F5F0E2",  
  creamDark: "#EAE2CC",  
  bark: "#8C3B2E",  
  brass: "#C9A227",  
  ink: "#2B2620",  
  inkSoft: "#6B6354",  
};
```

```
const QUESTIONS = [  
  {  
    id: "preference",  
    label: "Question 1",  
    text: "What is your preference for the three redwoods between Holes 4 and 5?"  
    options: [  
      { id: "keep", text: "Keep all three trees" },  
      { id: "remove", text: "Remove all three trees" },  
      { id: "partial", text: "Selective removal or thinning (compromise)" },  
      { id: "undecided", text: "Undecided – need more information" },  
    ],  
  },  
  {  
    id: "playability",  
    label: "Question 2",  
    text: "How do the redwoods affect your play of Holes 4 and 5?",  
    options: [  
      { id: "strategic", text: "They add a welcome strategic challenge" },  
      { id: "penal", text: "They are unfairly penal" },  
      { id: "safety", text: "They serve as a safety buffer between fairways" },  
      { id: "neutral", text: "Little effect on my play" },  
    ],  
  },  
];
```

```

{
  id: "compromise",
  label: "Question 3",
  text: "Would you support a negotiated compromise with the neighboring homeowners?",
  options: [
    { id: "yes", text: "Yes – open to a reasonable compromise" },
    { id: "no_keep", text: "No – the trees should remain untouched" },
    { id: "no_remove", text: "No – full removal is the right outcome" },
    { id: "defer", text: "Defer to the Green Committee's judgment" },
  ],
},
];

const STORAGE_PREFIX = "bcc-redwood-vote:";

function uid() {
  return Date.now().toString(36) + "-" + Math.random().toString(36).slice(2, 8);
}

export default function RedwoodPoll() {
  const [screen, setScreen] = useState("intro"); // intro | ballot | done | results
  const [answers, setAnswers] = useState({});
  const [memberName, setMemberName] = useState("");
  const [comment, setComment] = useState("");
  const [submitting, setSubmitting] = useState(false);
  const [submitError, setSubmitError] = useState("");
  const [results, setResults] = useState(null);
  const [loadingResults, setLoadingResults] = useState(false);
  const [hasVotedLocal, setHasVotedLocal] = useState(false);

  const allAnswered = QUESTIONS.every((q) => answers[q.id]);

  async function submitBallot() {
    if (!allAnswered || submitting) return;
    setSubmitting(true);
    setSubmitError("");
    const record = {
      answers,
      name: memberName.trim(),
      comment: comment.trim(),
      ts: new Date().toISOString(),
    };
    try {
      const res = await window.storage.set(
        STORAGE_PREFIX + uid(),
        JSON.stringify(record),
        true
      );
    } catch (error) {
      console.error("Error saving ballot:", error);
    }
  }
}

```

```

    );
    if (!res) throw new Error("Storage returned no result");
    setHasVotedLocal(true);
    setScreen("done");
  } catch (e) {
    setSubmitError(
      "Your ballot could not be recorded. Check your connection and press Submi
    );
  } finally {
    setSubmitting(false);
  }
}

async function loadResults() {
  setLoadingResults(true);
  setScreen("results");
  try {
    let keys = [];
    try {
      const listed = await window.storage.list(STORAGE_PREFIX, true);
      keys = listed?.keys || [];
    } catch {
      keys = [];
    }
    const records = await Promise.all(
      keys.map(async (k) => {
        try {
          const r = await window.storage.get(k, true);
          return r ? JSON.parse(r.value) : null;
        } catch {
          return null;
        }
      })
    );
    const valid = records.filter(Boolean);
    const tally = {};
    QUESTIONS.forEach((q) => {
      tally[q.id] = {};
      q.options.forEach((o) => (tally[q.id][o.id] = 0));
    });
    const comments = [];
    valid.forEach((r) => {
      QUESTIONS.forEach((q) => {
        const a = r.answers?.[q.id];
        if (a && tally[q.id][a] !== undefined) tally[q.id][a] += 1;
      });
      if (r.comment)

```

```

        comments.push({ name: r.name || "Member", text: r.comment, ts: r.ts });
    });
    comments.sort((a, b) => (a.ts < b.ts ? 1 : -1));
    setResults({ total: valid.length, tally, comments });
} catch {
    setResults({ total: 0, tally: null, comments: [], error: true });
} finally {
    setLoadingResults(false);
}
}

return (
    <div style={styles.page}>
        <style>{globalCss}</style>

        {/* Card header – set like the top of a club scorecard */}
        <header style={styles.header}>
            <div style={styles.headerRule} />
            <div style={styles.crestLine}>BERKELEY COUNTRY CLUB · EL CERRITO, CALIFOR
            <h1 style={styles.title}>The Redwoods at Four & Five</h1>
            <div style={styles.subtitle}>Member Ballot & Standing Tally</div>
            <div style={styles.headerRule} />
        </header>

        <main style={styles.card}>
            {screen === "intro" && (
                <Intro
                    onVote={() => setScreen("ballot")}
                    onResults={loadResults}
                    hasVoted={hasVotedLocal}
                />
            )}

            {screen === "ballot" && (
                <Ballot
                    answers={answers}
                    setAnswers={setAnswers}
                    memberName={memberName}
                    setMemberName={setMemberName}
                    comment={comment}
                    setComment={setComment}
                    allAnswered={allAnswered}
                    submitting={submitting}
                    submitError={submitError}
                    onSubmit={submitBallot}
                    onBack={() => setScreen("intro")}
                />
            )}
        </main>
    </div>
);

```

```

    })

    {screen === "done" && (
      <div style={{ textAlign: "center", padding: "32px 8px" }}>
        <div style={styles.doneMark}>✓</div>
        <h2 style={styles.h2}>Ballot recorded</h2>
        <p style={styles.body}>
          Thank you. Your response has been added to the standing tally for
          the Green Committee's review.
        </p>
        <button className="btn-solid" onClick={loadResults}>
          View standing tally
        </button>
      </div>
    )}

    {screen === "results" && (
      <Results
        results={results}
        loading={loadingResults}
        onRefresh={loadResults}
        onBack={() => setScreen("intro")}
      />
    )}
  </main>

  <footer style={styles.footer}>
    Responses are stored in a shared tally visible to all members using this
    poll. Names and comments are optional.
  </footer>
</div>
);
}

/* — Intro ————— */
function Intro({ onVote, onResults, hasVoted }) {
  return (
    <div>
      <p style={styles.eyebrow}>The matter before the membership</p>
      <p style={styles.body}>
        Three redwoods planted roughly forty years ago stand between the fairways
        of <strong>Hole 4</strong> and <strong>Hole 5</strong>. They were not par
        of the original course design. A neighboring homeowner has asked the Club
        to remove them, citing obstruction of a bay view under El Cerrito's view
        ordinance.
      </p>
      <p style={styles.body}>

```

Before the Club responds, the Green Committee is gathering member sentiment on the trees' value to the course — strategically, visually, and as a safety buffer between the two holes. This ballot takes about a minute. Three questions, plus an optional comment.

```
</p>
<SiteDiagram />
<div style={styles.btnRow}>
  <button className="btn-solid" onClick={onVote}>
    {hasVoted ? "Cast another ballot" : "Cast your ballot"}
  </button>
  <button className="btn-ghost" onClick={onResults}>
    View standing tally
  </button>
</div>
</div>
);
}

/* — Site diagram ————— */
/* Stylized elevation: neighbor's home on the hill at right, the
   three disputed redwoods between the fairways of 4 and 5, the
   two original cypresses, and the sightline toward the bay. */

function Redwood({ x, base, h, w, highlight }) {
  // Conical redwood built from stacked triangles + trunk
  const tiers = 4;
  const shapes = [];
  for (let i = 0; i < tiers; i++) {
    const tierW = w * (1 - i * 0.18);
    const tierTop = base - h * (0.38 + i * 0.17);
    const tierBot = base - h * (0.08 + i * 0.17);
    shapes.push(
      <polygon
        key={i}
        points={`${x},${tierTop} ${x - tierW / 2},${tierBot} ${x + tierW / 2},${t
          fill="#2E5238"
      />
    );
  }
  return (
    <g>
      <rect x={x - 2.5} y={base - h * 0.12} width={5} height={h * 0.12} fill="#6B
      {shapes}
      {highlight && (
        <ellipse
          cx={x}
          cy={base - h * 0.5}
```

```

        rx={w * 0.62}
        ry={h * 0.55}
        fill="none"
        stroke="#B33A28"
        strokeWidth="2.5"
        strokeDasharray="none"
    />
  })
</g>
);
}

function Cypress({ x, base, h, w }) {
  // Rounded, layered cypress silhouette
  return (
    <g>
      <rect x={x - 2.5} y={base - h * 0.15} width={5} height={h * 0.15} fill="#6B
      <ellipse cx={x} cy={base - h * 0.32} rx={w * 0.55} ry={h * 0.22} fill="#416
      <ellipse cx={x - w * 0.18} cy={base - h * 0.55} rx={w * 0.42} ry={h * 0.18}
      <ellipse cx={x + w * 0.12} cy={base - h * 0.74} rx={w * 0.3} ry={h * 0.15}
    </g>
  );
}

function SiteDiagram() {
  return (
    <figure style={styles.figure}>
      <svg
        viewBox="0 0 560 250"
        role="img"
        aria-label="Diagram: the neighbor's home sits uphill above Hole 4 at left
        style={{ width: "100%", height: "auto", display: "block" }}
      >
        {/** Sky */}
        <rect x="0" y="0" width="560" height="250" fill="#DCE8EC" />

        {/** Bay water at far right */}
        <rect x="410" y="118" width="150" height="60" fill="#9FBECC" />
        {/** Marin headland */}
        <path d="M440,118 L505,96 L560,118 Z" fill="#B6C6BD" />
        {/** Golden Gate tower suggestion */}
        <g stroke="#B33A28" strokeWidth="3" fill="none">
          <line x1="456" y1="118" x2="456" y2="92" />
          <line x1="482" y1="118" x2="482" y2="88" />
          <path d="M438,110 Q469,82 500,108" strokeWidth="2" />
        </g>

```

```

    { /* Hillside ground: rises right-to-left toward the neighbor */}
    <path
      d="M0,92 L160,128 Q260,168 410,178 L560,178 L560,250 L0,250 Z"
      fill="#5E8A52"
    />
    { /* Fairway bands: Hole 4 uphill at left, Hole 5 lower at right */}
    <path d="M120,120 Q170,134 230,148 L230,160 Q170,145 120,132 Z" fill="#7F
    <path d="M260,156 Q320,171 400,176 L400,184 Q320,180 260,168 Z" fill="#7F

    { /* Hole labels */}
    <text x="168" y="172" textAnchor="middle" style={{ font: "italic 600 13px
      Hole 4
    </text>
    <text x="318" y="204" textAnchor="middle" style={{ font: "italic 600 13px
      Hole 5
    </text>

    { /* 4th green, uphill at left, with flag */}
    <ellipse cx="136" cy="124" rx="24" ry="6" fill="#8FBF74" />
    <g>
      <line x1="136" y1="124" x2="136" y2="94" stroke="#F5F0E2" strokeWidth="
      <polygon points="136,94 154,100 136,106" fill="#B33A28" />
    </g>

    { /* Hole 5 flag, downhill at right */}
    <g>
      <line x1="392" y1="178" x2="392" y2="148" stroke="#F5F0E2" strokeWidth=
      <polygon points="392,148 410,154 392,160" fill="#B33A28" />
    </g>

    { /* The two original cypresses – left of the redwoods, nearer the
      4th green, roughly half again taller than the redwoods */}
    <Cypress x={196} base={146} h={168} w={42} />
    <Cypress x={224} base={150} h={156} w={38} />

    { /* The three disputed redwoods, circled */}
    <Redwood x={248} base={155} h={110} w={42} highlight />
    <Redwood x={276} base={160} h={132} w={50} highlight />
    <Redwood x={304} base={166} h={118} w={46} highlight />

    { /* Neighbor's home on the hill above Hole 4 */}
    <g>
      <rect x="26" y="74" width="52" height="30" fill="#F5F0E2" stroke="#2B26
      <polygon points="22,74 52,56 82,74" fill="#8C3B2E" stroke="#2B2620" str
      <rect x="58" y="84" width="10" height="20" fill="#6B6354" />
      <rect x="36" y="84" width="12" height="10" fill="#9FBECC" stroke="#2B26
    </g>

```



```

        <text x="52" y="48" textAnchor="middle" style={{ font: "600 12px Georgia,
        Neighbor
    </text>

    /* Sightline from home toward the bay, interrupted by redwoods */
    <line
        x1="72" y1="82" x2="445" y2="104"
        stroke="#B33A28" strokeWidth="2" strokeDasharray="7 6" opacity="0.85"
    />
    <text x="330" y="92" style={{ font: "italic 12px Georgia, serif", fill: "
        view to the bay
    </text>
</svg>
<figcaption style={styles.figCaption}>
    Stylized elevation, not to scale. The neighbor's home sits uphill above
    Hole 4; circled are the three redwoods (planted c.&nbsp;1985) between
    the fairways of Holes 4 and 5. Uphill of the redwoods, nearer the 4th
    green, stand the two original cypresses – roughly half again the
    redwoods' height – with the sightline running toward the bay and
    Golden Gate below.
</figcaption>
</figure>
    );
}

/* — Ballot ————— */
function Ballot({
    answers,
    setAnswers,
    memberName,
    setMemberName,
    comment,
    setComment,
    allAnswered,
    submitting,
    submitError,
    onSubmit,
    onBack,
}) {
    return (
        <div>
            {QUESTIONS.map((q, qi) => (
                <fieldset key={q.id} style={styles.fieldset}>
                    <div style={styles.qLabel}>{q.label}</div>
                    <legend style={styles.qText}>{q.text}</legend>
                    <div role="radiogroup" aria-label={q.text}>
                        {q.options.map((o) => {

```

```

const selected = answers[q.id] === o.id;
return (
  <button
    key={o.id}
    className={"opt" + (selected ? " opt-on" : "")}
    role="radio"
    aria-checked={selected}
    onClick={() =>
      setAnswers((a) => ({ ...a, [q.id]: o.id }))
    }
  >
    <span className="opt-dot" aria-hidden="true" />
    {o.text}
  </button>
);
}}}
</div>
{qi < QUESTIONS.length - 1 && <div style={styles.divider} />}
</fieldset>
)}}

<div style={styles.divider} />

<label style={styles.inputLabel}>
  Name or member number <span style={styles.optionalTag}>optional</span>
  <input
    style={styles.input}
    value={memberName}
    onChange={(e) => setMemberName(e.target.value)}
    placeholder="e.g., J. Whitfield · #214"
    maxLength={60}
  />
</label>

<label style={styles.inputLabel}>
  Comment for the Green Committee{" "}
  <span style={styles.optionalTag}>optional</span>
  <textarea
    style={{ ...styles.input, minHeight: 84, resize: "vertical" }}
    value={comment}
    onChange={(e) => setComment(e.target.value)}
    placeholder="Anything the committee should weigh – pace of play, shade,
    maxLength={600}
  />
</label>

{submitError && <div style={styles.error}>{submitError}</div>}

```

```

<div style={styles.btnRow}>
  <button
    className="btn-solid"
    onClick={onSubmit}
    disabled={!allAnswered || submitting}
  >
    {submitting ? "Recording..." : "Submit ballot"}
  </button>
  <button className="btn-ghost" onClick={onBack} disabled={submitting}>
    Back
  </button>
</div>
{!allAnswered && (
  <div style={styles.hint}>Answer all three questions to submit.</div>
)}
</div>
);
}

/* — Results ————— */
function Results({ results, loading, onRefresh, onBack }) {
  if (loading) {
    return (
      <div style={{ textAlign: "center", padding: "40px 0" }}>
        <div className="spinner" aria-label="Loading results" />
        <p style={{ ...styles.body, marginTop: 16 }}>Tallying ballots...</p>
      </div>
    );
  }
  if (!results) return null;

  if (results.error) {
    return (
      <div>
        <h2 style={styles.h2}>Tally unavailable</h2>
        <p style={styles.body}>
          The standing tally could not be loaded. Press Refresh to try again.
        </p>
        <div style={styles.btnRow}>
          <button className="btn-solid" onClick={onRefresh}>Refresh tally</button>
          <button className="btn-ghost" onClick={onBack}>Back</button>
        </div>
      </div>
    );
  }
}

```

```

return (
  <div>
    <div style={styles.tallyHead}>
      <h2 style={{ ...styles.h2, margin: 0 }}>Standing tally</h2>
      <span style={styles.tallyCount}>
        {results.total} ballot{results.total === 1 ? "" : "s"} cast
      </span>
    </div>

    {results.total === 0 && (
      <p style={styles.body}>
        No ballots yet. Be the first – cast your ballot from the opening page.
      </p>
    )}

    {results.total > 0 &&
      QUESTIONS.map((q) => {
        const counts = results.tally[q.id];
        const max = Math.max(1, ...Object.values(counts));
        return (
          <div key={q.id} style={{ marginBottom: 28 }}>
            <div style={styles.qLabel}>{q.label}</div>
            <div style={{ ...styles.qText, marginBottom: 12 }}>{q.text}</div>
            {q.options.map((o) => {
              const n = counts[o.id];
              const pct = results.total
                ? Math.round((n / results.total) * 100)
                : 0;
              return (
                <div key={o.id} style={styles.barRow}>
                  <div style={styles.barLabel}>{o.text}</div>
                  <div style={styles.barTrack}>
                    <div
                      className="bar-fill"
                      style={{
                        width: `${(n / max) * 100}%`,
                        background:
                          n === max && n > 0 ? C.brass : C.pine,
                      }}
                    />
                  </div>
                  <div style={styles.barNum}>
                    {n} · {pct}%
                  </div>
                </div>
              );
            })}
          </div>
        );
      })}
    )}
  )}

```

```

        </div>
    );
    }}}

    {results.comments.length > 0 && (
    <div>
        <div style={styles.divider} />
        <div style={styles.qLabel}>Member comments</div>
        {results.comments.map((c, i) => (
            <div key={i} style={styles.comment}>
                <div style={styles.commentName}>{c.name}</div>
                <div style={styles.commentText}>{c.text}</div>
            </div>
        ))}
    </div>
    )}

    <div style={styles.btnRow}>
        <button className="btn-solid" onClick={onRefresh}>Refresh tally</button>
        <button className="btn-ghost" onClick={onBack}>Back</button>
    </div>
</div>
);
}

/* — Styles ————— */
const styles = {
  page: {
    minHeight: "100vh",
    background: `linear-gradient(180deg, ${C.pineDeep} 0%, ${C.pine} 100%)`,
    fontFamily: "'Source Serif 4', Georgia, 'Times New Roman', serif",
    color: C.ink,
    padding: "0 16px 40px",
  },
  header: {
    maxWidth: 640,
    margin: "0 auto",
    padding: "36px 4px 24px",
    textAlign: "center",
    color: C.cream,
  },
  headerRule: {
    height: 1,
    background: `linear-gradient(90deg, transparent, ${C.brass}, transparent)`,
    margin: "10px 0",
  },
  crestLine: {

```

```
    fontFamily: "'Source Serif 4', Georgia, serif",
    fontSize: 11,
    letterSpacing: "0.28em",
    color: C.brass,
    margin: "6px 0 10px",
  },
  title: {
    fontFamily: "'Playfair Display', Georgia, serif",
    fontWeight: 700,
    fontSize: "clamp(30px, 6vw, 44px)",
    lineHeight: 1.1,
    margin: "0 0 8px",
    color: C.cream,
  },
  subtitle: {
    fontStyle: "italic",
    fontSize: 15,
    color: "#CFE0CF",
    marginBottom: 10,
  },
  card: {
    maxWidth: 640,
    margin: "0 auto",
    background: C.cream,
    border: `1px solid ${C.creamDark}`,
    borderTop: `4px solid ${C.bark}`,
    boxShadow: "0 18px 40px rgba(0,0,0,0.35)",
    padding: "28px 26px 30px",
  },
  footer: {
    maxWidth: 640,
    margin: "14px auto 0",
    textAlign: "center",
    fontSize: 12.5,
    fontStyle: "italic",
    color: "#A9BFA9",
  },
  eyebrow: {
    fontSize: 11,
    letterSpacing: "0.22em",
    textTransform: "uppercase",
    color: C.bark,
    fontWeight: 600,
    margin: "0 0 10px",
  },
  body: {
    fontSize: 16.5,
```

```

    lineHeight: 1.65,
    margin: "0 0 14px",
  },
  h2: {
    fontFamily: "'Playfair Display', Georgia, serif",
    fontSize: 26,
    margin: "0 0 10px",
  },
  fieldset: { border: "none", margin: 0, padding: 0 },
  qLabel: {
    fontSize: 11,
    letterSpacing: "0.22em",
    textTransform: "uppercase",
    color: C.bark,
    fontWeight: 600,
    marginBottom: 4,
  },
  qText: {
    fontFamily: "'Playfair Display', Georgia, serif",
    fontSize: 19,
    fontWeight: 700,
    lineHeight: 1.35,
    marginBottom: 12,
    padding: 0,
  },
  divider: {
    height: 1,
    background: C.creamDark,
    margin: "22px 0",
  },
  figure: {
    margin: "6px 0 20px",
    border: `1px solid ${C.creamDark}`,
    background: "#FDFBF4",
    padding: 8,
  },
  figCaption: {
    fontSize: 12.5,
    fontStyle: "italic",
    lineHeight: 1.5,
    color: C.inkSoft,
    padding: "8px 6px 2px",
  },
  inputLabel: {
    display: "block",
    fontSize: 14,
    fontWeight: 600,

```

```

    margin: "0 0 16px",
  },
  optionalTag: {
    fontWeight: 400,
    fontStyle: "italic",
    color: C.inkSoft,
    fontSize: 13,
  },
  input: {
    display: "block",
    width: "100%",
    boxSizing: "border-box",
    marginTop: 6,
    padding: "10px 12px",
    fontSize: 15,
    fontFamily: "inherit",
    color: C.ink,
    background: "#FDFBF4",
    border: `1px solid ${C.creamDark}`,
    borderRadius: 2,
  },
  btnRow: {
    display: "flex",
    gap: 12,
    flexWrap: "wrap",
    marginTop: 18,
  },
  hint: {
    marginTop: 10,
    fontSize: 13.5,
    fontStyle: "italic",
    color: C.inkSoft,
  },
  error: {
    marginTop: 8,
    padding: "10px 12px",
    background: "#F7E5E1",
    border: `1px solid ${C.bark}`,
    color: C.bark,
    fontSize: 14,
  },
  doneMark: {
    width: 54,
    height: 54,
    margin: "0 auto 14px",
    borderRadius: "50%",
    border: `2px solid ${C.brass}`,
  },

```



```

    color: C.pine,
    fontSize: 26,
    lineHeight: "52px",
  },
  tallyHead: {
    display: "flex",
    alignItems: "baseline",
    justifyContent: "space-between",
    gap: 10,
    marginBottom: 20,
    flexWrap: "wrap",
  },
  tallyCount: {
    fontSize: 13,
    letterSpacing: "0.12em",
    textTransform: "uppercase",
    color: C.inkSoft,
  },
  barRow: {
    display: "grid",
    gridTemplateColumns: "minmax(120px, 1fr) minmax(80px, 1.1fr) auto",
    alignItems: "center",
    gap: 10,
    marginBottom: 8,
  },
  barLabel: { fontSize: 14, lineHeight: 1.3 },
  barTrack: {
    height: 14,
    background: C.creamDark,
    overflow: "hidden",
  },
  barNum: {
    fontSize: 13,
    fontVariantNumeric: "tabular-nums",
    color: C.inkSoft,
    whiteSpace: "nowrap",
  },
  comment: {
    borderLeft: `3px solid ${C.brass}`,
    padding: "8px 12px",
    margin: "12px 0",
    background: "#FDFBF4",
  },
  commentName: {
    fontSize: 12.5,
    fontWeight: 700,
    letterSpacing: "0.06em",
  },

```

```

      textTransform: "uppercase",
      color: C.bark,
      marginBottom: 3,
    },
    commentText: { fontSize: 15, lineHeight: 1.5 },
  };

const globalCss = `
@import url('https://fonts.googleapis.com/css2?family=Playfair+Display:ital,wght@

.opt {
  display: flex;
  align-items: center;
  gap: 12px;
  width: 100%;
  text-align: left;
  font-family: inherit;
  font-size: 15.5px;
  line-height: 1.4;
  color: ${C.ink};
  background: transparent;
  border: 1px solid ${C.creamDark};
  border-radius: 2px;
  padding: 11px 14px;
  margin-bottom: 8px;
  cursor: pointer;
  transition: background 0.15s, border-color 0.15s;
}

.opt:hover { background: #EFE8D5; }
.opt:focus-visible, .btn-solid:focus-visible, .btn-ghost:focus-visible,
input:focus-visible, textarea:focus-visible {
  outline: 2px solid ${C.brass};
  outline-offset: 2px;
}

.opt-dot {
  flex: 0 0 14px;
  width: 14px; height: 14px;
  border-radius: 50%;
  border: 2px solid ${C.pine};
  background: transparent;
  transition: background 0.15s;
}

.opt-on { border-color: ${C.pine}; background: #E7EFE7; }
.opt-on .opt-dot { background: ${C.pine}; }

.btn-solid, .btn-ghost {
  font-family: inherit;

```

```

    font-size: 15px;
    letter-spacing: 0.04em;
    padding: 11px 22px;
    border-radius: 2px;
    cursor: pointer;
    transition: background 0.15s, color 0.15s;
}
.btn-solid {
    background: ${C.pine};
    color: ${C.cream};
    border: 1px solid ${C.pine};
}
.btn-solid:hover:not(:disabled) { background: ${C.pineDeep}; }
.btn-solid:disabled { opacity: 0.45; cursor: not-allowed; }
.btn-ghost {
    background: transparent;
    color: ${C.pine};
    border: 1px solid ${C.pine};
}
.btn-ghost:hover:not(:disabled) { background: #E7EFE7; }

.bar-fill { height: 100%; transition: width 0.5s ease; }

.spinner {
    width: 30px; height: 30px;
    margin: 0 auto;
    border: 3px solid ${C.creamDark};
    border-top-color: ${C.pine};
    border-radius: 50%;
    animation: spin 0.9s linear infinite;
}
@keyframes spin { to { transform: rotate(360deg); } }
@media (prefers-reduced-motion: reduce) {
    .bar-fill { transition: none; }
    .spinner { animation-duration: 2s; }
}
`;

```